

Cost-Distance Steiner Trees for Timing-Constrained Global Routing

Stephan Held and Edgar Perner

Research Institute for Discrete Mathematics and Hausdorff Center for Mathematics

University of Bonn

Bonn, Germany

held@dm.uni-bonn.de and perner@dm.uni-bonn.de

Abstract—The cost-distance Steiner tree problem seeks a Steiner tree that minimizes the total congestion cost plus the weighted sum of source-sink delays. This problem arises as a subroutine in timing-constrained global routing with a linear delay model, used before buffer insertion. Here, the congestion cost and the delay of an edge are essentially uncorrelated, unlike in most other algorithms for timing-driven Steiner trees.

We present a fast algorithm for the cost-distance Steiner tree problem. Its running time is $\mathcal{O}(t(n \log n + m))$, where t , n , and m are the numbers of terminals, vertices, and edges in the global routing graph. We also prove that our algorithm guarantees an approximation factor of $\mathcal{O}(\log t)$. This matches the best-known approximation factor for this problem, but with a much faster running time.

To account for increased capacitance and delays after buffering caused by bifurcations, we incorporate a delay penalty for each bifurcation without compromising the running time or approximation factor.

In our experimental results, we show that our algorithm outperforms previous methods that first compute a Steiner topology, e.g. based on shallow-light Steiner trees or the Prim-Dijkstra algorithm, and then embed this into the global routing graph.

Index Terms—Cost-distance Steiner tree, timing-constrained global routing

I. INTRODUCTION

Modern routers do not only aim for minimum total wire length, but need to consider signal timing as well [1], [8], [13], [15], [20]–[22]. Before and in preparation of buffer insertion, this is usually estimated using a linear delay model, where the delay depends linearly on the wire length, as well as on wire type and layer [4], [18]. There are many Steiner tree variants that take path lengths into account [2]–[4], [6], [14]. They do not consider independent congestion and delay costs.

The cost-distance Steiner tree problem overcomes this limitation. It asks for a Steiner tree minimizing the sum of routing congestion cost and weighted path delays through the tree. It occurs as the central sub-problem in timing-constrained global routing, where it is usually solved heuristically [13].

We are given a graph G with edge cost $c : E(G) \rightarrow \mathbb{R}_+$ and edge delays $d : E(G) \rightarrow \mathbb{R}_+$. In our application G is a 3D global routing graph, where an edge cost $c(e)$ arises from the current edge usage and $d(e)$ is the delay of e in a linear delay model. If multiple wire types (width and spacing configurations) are available G may have a parallel edge for each wire type that has an individual cost and delay.

Given a net connecting a set of sinks S to a root r , delay weights $w : \mathbb{R}_+$ and positions $\pi : S \cup \{r\} \rightarrow V(G)$, we seek for a Steiner tree T connecting $S \cup \{r\}$ that is embedded into G , i.e. has an extension $\pi : V(T) \setminus (S \cup \{r\}) \rightarrow V(G)$ and minimizes

$$\text{cost}(T) = \sum_{e \in T} c(e) + \sum_{t \in S} w(t) \text{delay}_T(r, t), \quad (1)$$

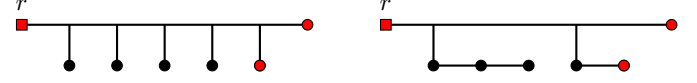


Fig. 1. Two trees for the same net, the right one with fewer bifurcations on the path from the root r to critical sinks (red dots).

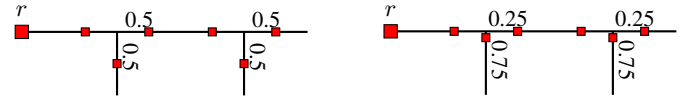


Fig. 2. Two buffering solutions (small red squares) indicating different delay tradeoffs between at the branchings.

where $\text{delay}_T(r, t) = \sum_{e \in E(T_{[r, t]})} d(e)$ in a simple linear delay model. The delay weights indicate the criticality of a sink and arise from a Lagrangean relaxation in timing-constrained global routing [13].

Throughout this paper, we use $t := |S \cup \{r\}|$, $n := |V(G)|$ and $m := |E(G)|$. The best known approximation factor for this problem is $\mathcal{O}(\log t)$ based on iteratively computing a matching of terminals that are served by a Steiner vertex based on a special distance function [5], [17] with a running time of $\mathcal{O}(t^2(n \log n + m))$. It does not permit an approximation factor better than $\Omega(\log \log t)$ unless $\text{NP} \subseteq \text{DTIME}(t^{\mathcal{O}(\log \log \log t)})$ [7]. For the uniform case, where $c = d$, constant approximation factors are possible [9], [17].

Bifurcations on a path increase the capacitance and delay after buffering. In Figure 1, we see a critical path with many bifurcations on the left, which is unfavorable to the tree on the right after buffering. To mitigate this, we use a delay penalty for each bifurcation, as proposed in [4].

As buffering has some freedom to shield off capacitance (see Figure 2), we use a model where the bifurcation penalty can be distributed within certain limits. For technical reasons, we assume that the root and the sinks form the leaves of T and internal vertices have degree at most 3. We say that such a tree is *bifurcation compatible*. As we allow multiple vertices with the same position, any Steiner tree can be transformed into such a tree without changing the total length or any source-sink length in the tree.

We can consider T as an r -arborescence. Then, at every proper Steiner vertex u with $\delta_T^+(u) = \{x, y\}$ we distribute a delay penalty to (u, x) and (u, y) as follows. Let $d_{bif} \in \mathbb{R}_+$ be the total bifurcation penalty for both branches and $0 \leq \eta \leq 1/2$. For $\lambda_x \in [\eta, 1 - \eta]$, we assign a delay penalty of $\lambda_x d_{bif}$ to (u, x) and $\lambda_y d_{bif}$ to (u, y) with $\lambda_y = (1 - \lambda_x)$.

In the cost-distance objective with weighted delays, the optimum choice of λ_x depends only on the total delay weights in the two

subtrees:

$$\lambda_x = \begin{cases} \eta & \text{if } w(S \cap V(T_x)) > w(S \cap V(T_y)) \\ 0.5 & \text{if } w(S \cap V(T_x)) = w(S \cap V(T_y)) \\ (1 - \eta) & \text{else.} \end{cases} \quad (2)$$

If $w(S \cap V(T_x)) = w(S \cap V(T_y))$ any choice is optimal. T_x is the sub-arborescence rooted at x and $w(X) = \sum_{x \in X} w(x)$. In [4], the special case $\eta = 0.5$ was considered. Minimizing the maximum delay would lead to different choices of λ_x .

Setting λ_x according to (2) for every edge (u, x) with $|\delta^+(u)| = 2$ and $\lambda_x = 0$ elsewhere, we have the following delay model, which we assume for (1) from now on:

$$\text{delay}_T(r, t) = \sum_{e=(u,v) \in E(T_{[r,t]})} (d(e) + \lambda_v d_{bif}). \quad (3)$$

The parameter d_{bif} is computed similarly to [4] by taking an optimally spaced uniform repeater chain. Then, d_{bif} is the delay increase when adding the input capacitance in the middle of a single net, minimizing over all layers and wire types.

A. Our Contributions

Our contributions are the following:

- 1) A new randomized $\mathcal{O}(\log t)$ -approximation algorithm for cost-distance Steiner trees with bifurcation penalties (Section II). Its running time of $\mathcal{O}(t(n \log n + m))$ improves by a factor of t over [17].
- 2) Multiple enhancements to improve its practical performance (Section III).
- 3) Experimental comparisons with the state-of-the-art heuristics, demonstrating a significant benefit of our method in timing-constrained global routing (Section IV).

II. A FAST APPROXIMATION ALGORITHM

Our algorithm (Algorithm 1) works similar to Kruskal's algorithm for spanning trees [16]. It builds a Steiner tree by iteratively merging components. In each iteration i , we propagate distance labels from all sinks simultaneously but with a special sink-specific distance function (see Theorem 1) until a permanent label reaches another terminal. This determines the vertices u and v in Line 3, which are joined through a new Steiner point s that replaces u and v in the terminal set S_{i+1} of the next iteration and has the sum of their delay weights as its own weight. Note that the component containing the root vertex r_i might also be merged. In this case, s is inserted at the position $\pi(r_i)$ and serves as the root r_{i+1} in the next iteration (line 5). If two sinks are merged, we select the position $\pi(s)$ randomly proportional to their delay weight (line 7). In the end, the inserted Steiner vertices are the bifurcations of the tree.

A key difference to [16] is the cost function. In [17], the main algorithm uses for each pair u, v of sinks an individual distance function $c(e) + 2w(u)w(v)/(w(u) + w(v)) \cdot d(e)$ ($e \in E(G)$). It combines costs and weighted delays. Computing shortest paths between each pair of terminals with an individual cost function would be prohibitively slow in a large global routing graph. We use one distance function

$$l_u(e) := c(e) + w(u) \cdot d(e) \quad (e \in E(G)) \quad (4)$$

per sink u as in [5]. While the algorithms in [5], [17] merge iteratively based on a matching involving (a constant fraction of) all terminals, our algorithm allows the newly inserted Steiner vertex to be merged in the next iteration, before any other merges take place.

We address the bifurcation delay penalties as follows. Let

$$\beta(w, w') = d_{bif} (\eta \max(w, w') + (1 - \eta) \min(w, w'))$$

Algorithm 1: Cost-Distance Algorithm

Input: An instance $(G, S, r, w, c, d, d_{bif}, \eta)$ of the Cost-Distance Steiner tree problem

Output: An embedded Steiner tree (T, π) on S with root r .

```

1  $T \leftarrow (S \cup \{r\}, \emptyset)$ ,  $i \leftarrow 0$ ,  $S_0 \leftarrow S$ ,  $r_0 \leftarrow r$ .
2 while  $|S_i| > 0$  do
3   Find the nodes  $u \in S_i$ ,  $v \in S_i \cup \{r_i\}$  minimizing  $L(u, v)$ 
   (see (5)& Theorem 1).
4   if  $v = r_i$  then
5     Add a new Steiner vertex  $s$  placed at  $\pi(s) = \pi(r_i)$ ,
      $r_{i+1} \leftarrow s$ ,  $w(s) \leftarrow w(u)$ ,  $S_{i+1} \leftarrow S_i \setminus \{u\}$ .
6   else
7     Randomly select a position  $p \in \{\pi(u), \pi(v)\}$  with
     probabilities proportional to their delay weight. Add a
     new Steiner vertex  $s$  at  $\pi(s) = p$  with
      $w(s) \leftarrow w(u) + w(v)$ ,  $S_{i+1} \leftarrow S_i \cup \{s\} \setminus \{u, v\}$ .
8   Add the  $u$ - $s$ - $v$  connection corresponding to the path
   underlying  $L(u, v)$  to  $T$ .
9    $i \leftarrow i + 1$ .
10 Return the embedded tree  $(T, \pi)$ .
```

denote the minimum possible weighted delay penalty when merging two sinks with delay weights w and w' . Then, the weighted bifurcation delay penalty for merging $u \in S_i$ and $v \in S_i \cup \{r_i\}$ is

$$b(u, v) := \begin{cases} \beta(w(u), w(v)) & (v \in S_i) \\ \beta(w(u), w(S_i \setminus \{u\})) & (v = r_i) \end{cases}.$$

In line 3 of Algorithm 1, the pair u, v minimizing

$$L(u, v) := \begin{cases} \text{dist}_{G, c + \min\{w(u), w(v)\} \cdot d}(u, v) + b(u, v) & (v \in S_i) \\ \text{dist}_{G, c + w(u) \cdot d}(u, r_i) + b(u, r_i) & (v = r_i) \end{cases} \quad (5)$$

is chosen. By $\text{dist}_{\Gamma, \gamma}(x, y)$ we denote the shortest path distance between x and y in the graph Γ w.r.t. edge lengths γ . $L(r_i, v) := L(v, r_i)$ completes the definition of L . We can still prove an approximation factor of $\mathcal{O}(\log t)$. In addition, our algorithm is substantially faster than [5], [17].

A. An Example

Figure 3 visualizes the course of the algorithm for a set of 5 sinks, the black dots in the first picture. The size of the dots represents their delay weight. We call the elements of S_i in iteration i the *active* vertices (marked blue). In iteration $i = 0$ of the algorithm, $S_0 = S$, thus all sinks are active, and we start Dijkstra searches (areas inside blue circles) from all sinks using their individual cost functions. The larger the delay weight, the slower the circle grows. The terminal u is the first one to find a connection (to v).

One of $\pi(u)$ and $\pi(v)$ is now chosen randomly with probability proportional to its delay weight to place the Steiner vertex s with weight $w(s) := w(u) + w(v)$. The random choice makes it more likely for the heavier weighted terminal to be reachable from r without a detour. Here, the more likely vertex v was chosen.

The shortest u - s - v path regarding l_u (red) is added to T . The terminals u and v are deactivated and s is activated. In the next iteration $i = 1$, we started a new Dijkstra search from s w.r.t. l_s . In this iteration the shortest connection is a root connection, so we do not make a random choice for our Steiner vertex and just deactivate the sink. We continue connecting vertices and decreasing the number of active vertices until all sinks are connected.

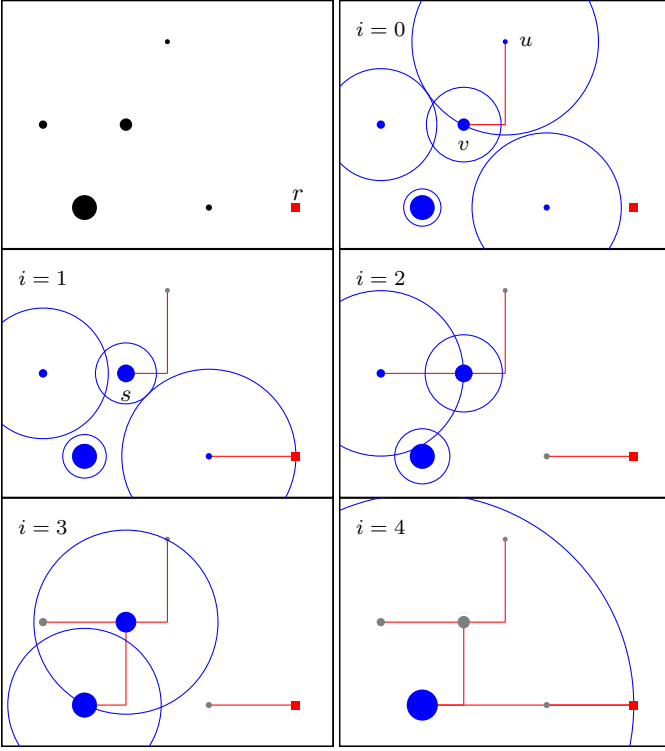


Fig. 3. Cost-distance algorithm example. Root in red, the sinks S in black. For simplicity the labeled vertices in global routing graph are visualized as Euclidean balls.

B. Theoretical Properties

We first analyze the running time of the algorithm.

Theorem 1. *The cost-distance algorithm can be implemented with a running time of $\mathcal{O}(t(n \log n + m))$.*

Proof. The total running time is dominated by the shortest path computations in Step 3. We start a Dijkstra search from each sink $u \in S_i$ simultaneously using the individual distance function l_u .

Whenever we enter a vertex $v \in S_i \cup \{r_i\} \setminus \{u\}$, we add the optimally balanced weighted node delay $b(u, v)$. If such a vertex gets permanently labeled, we have found a pair u, v minimizing $L(u, v)$. After adding the connection, we start a Dijkstra search from the newly inserted Steiner vertex s . Dijkstra labels from other terminals remain valid in the next iteration. As we insert $|S| - 1$ Steiner vertices, we have $2|S| - 1$ path searches, implementing shortest path searches with Fibonacci heaps [10] yields the claimed running time. $\square$

We now prove the approximation guarantee of our algorithm. Throughout, let OPT denote the cost according to (1) and (3) of an optimum solution T^* to the original instance. The challenge lies in bounding the incremental cost increase from each iteration. Similar to [17], we show that there is a cheap matching of terminals w.r.t. weights L (Lemma 4) which covers roughly the price increase of $\frac{1}{4}$ of the iterations (Theorem 6).

In iteration i , consider the potential

$$D_i := \sum_{s \in S_i} w(s) \text{delay}_{T^*}(r, s).$$

As new Steiner vertices $s \in S_i$ are mapped to a position $\pi(s) = \pi(v')$, $v' \in S \cup \{r\}$, $\text{delay}_{T^*}(r, s) := \text{delay}_{T^*}(r, v')$ is well-defined for all $s \in S_i$.

Lemma 2. *In each iteration i , there is a cost-distance tree for the terminal set $S_i \cup \{r_i\}$ with expected total cost at most*

$$C(T^*) + \mathbb{E}[D_i] \leq \text{OPT}.$$

Proof. Suppose that two sinks $u, v \in S_i$ are matched in iteration i . They contribute a delay cost of $w(u) \cdot \text{delay}_{T^*}(r, u) + w(v) \cdot \text{delay}_{T^*}(r, v)$ to D_i in iteration i . The expected contribution of the new Steiner vertex s to D_{i+1} is

$$\begin{aligned} & \sum_{x \in \{u, v\}} \frac{w(x)}{w(u) + w(v)} \cdot w(s) \cdot \text{delay}_{T^*}(r, x) \\ &= w(u) \cdot \text{delay}_{T^*}(r, u) + w(v) \cdot \text{delay}_{T^*}(r, v). \end{aligned}$$

Otherwise, if one of the merged nodes is the root, the new Steiner vertex does not contribute to D_{i+1} . This means that we have $\mathbb{E}[D_{i+1}] \leq \mathbb{E}[D_i]$. Thus, $C(T^*) + \mathbb{E}[D_i] \leq \text{OPT}$ by induction. As T^* can be used as a solution for S_i , we have proven the lemma. $\square$

A matching M in a set X is called *near-perfect* if it leaves at most one element uncovered, i.e. $|X \setminus \cup_{e \in M} e| \leq 1$.

Lemma 3. *Given a bifurcation compatible tree $T = (V, E)$ and a set of nodes $S \subseteq V$, there exists a near-perfect matching of the nodes in S such that the union of the unique paths in T between matched nodes contains each edge and each vertex of the tree at most once.*

Proof. Lemma 4.2 in [17] proves that for an arbitrary tree there is such a matching using each edge at most once. If T is bifurcation compatible, this implies that each vertex is contained in at most one path. $\square$

Lemma 4. *In each iteration i , there exists a near-perfect matching M of $S_i \cup \{r_i\}$ with $\mathbb{E}[L(M)] \leq \text{OPT}$.*

Proof. By Lemma 3, there is a near-perfect matching M in $S_i \cup \{r_i\}$ such that the union of the unique paths in T^* between matched nodes contains each edge and each vertex at most once.

For technical reasons, we extend w to $w(r_i)$ with an arbitrary constant. Given $\{u, v\} \in M$, we may assume w.l.o.g. $w(u) \leq w(v)$ or $v = r_i$. Then the cost of M is

$$\begin{aligned} & \sum_{(u,v) \in M} L(u, v) \\ & \leq \sum_{(u,v) \in M} \left(\text{dist}_{T^*,c}(u, v) + w(u) \text{dist}_{T^*,d}(u, v) + b(u, v) \right) \\ & \leq C(T^*) + \sum_{(u,v) \in M} \left(w(u) \text{dist}_{T^*,d}(r, u) + w(v) \text{dist}_{T^*,d}(r, v) \right. \\ & \quad \left. + b(u, v) \right) \leq C(T^*) + D_i, \end{aligned}$$

where the first inequality holds due to Lemma 3. The next inequality holds because we added twice the distance w.r.t d from r to the u - v path in T^* and $w(u) \leq w(v)$. The last inequality holds because $b(u, v)$ uses the optimum delay penalty distribution, and the unique paths defined by the matching use each vertex exactly once. In contrast, D_i uses a penalty distribution, where penalties can be counted multiple times. Observe that the $b(u, v)$ values are accounted for in D_i either at the unique least common ancestor (w.r.t. the root r) or along the vertices of an r - u -path.

By Lemma 2, $\mathbb{E}[C(T^*) + D_i] \leq \text{OPT}$. $\square$

Lemma 5. *When the algorithm merges $u \in S_i$ and $v \in S_i \cup \{r_i\}$ in iteration i , $L(u, v)$ reflects at least half of the expected objective cost increase in this iteration.*

Proof. $L(u, v)$ reflects the correct connection cost and bifurcation penalty $b(u, v)$. Let P be the underlying path reflecting the embedding into G . If $v \neq r_i$, the vertex $x \in \{u, v\}$ is chosen with a probability of $w(x)/(w(u) + w(v))$. Thus, the expected delay cost w.r.t. d is

$$\frac{2w(u)w(v)}{w(u) + w(v)} \text{dist}_{P,d}(u, v) \leq 2 \min\{w(u), w(v)\} \text{dist}_{P,d}(u, v)$$

and

$$2 \min\{w(u), w(v)\} \text{dist}_{P,d}(u, v) + \text{dist}_{P,c}(u, v) + b(u, v) \leq 2L(u, v)$$

If $v = r_i$, the (deterministic) cost is $L(u, v)$. $\square$

Theorem 6. *The cost-distance algorithm computes a Steiner tree with expected objective value at most $\mathcal{O}(\log t) \cdot \text{OPT}$.*

Proof. Consider a minimum weight near-perfect matching M in $(S \cup \{r\})$. Order the matching edges $M = \{\{u_1, v_1\}, \dots, \{u_{\lfloor t/2 \rfloor}, v_{\lfloor t/2 \rfloor}\}\}$ in non-decreasing order w.r.t. L .

For the first $\lfloor t/4 \rfloor$ iterations, the i -th (shortest) path from our algorithm will have costs at most $L(u_{2i-1}, v_{2i-1})$, because each path can intersect at most two matching edges. Thus, we paid at most half of the cost of an optimal near perfect matching when adding up the $\lfloor t/4 \rfloor$ path costs $L(u, v)$. By Lemmas 4 and 5, this implies a cost of at most OPT for reducing the size of our instance by a factor (almost) $3/4$.

Reducing the size repeatedly, we pay $\mathcal{O}(\log_{4/3} t)$ times OPT overall. $\square$

III. PRACTICAL ENHANCEMENTS

In this section, we present multiple practical enhancements.

A. Discounting Existing Tree Components

The variant from Section II inserts Steiner vertices only at terminal positions, which leads to unnecessary detours. To avoid this, we set $c(e) = 0$ for edges in the connected components of T where the current path search starts and ends, and allow reusing them. We add separate labels for each possible end component a path search reaches so that the zero connection cost edges can only be used to reach that particular end component. This implicitly places Steiner vertices at the points where the path leaves or enters the connected components. We still restart the next path search from one of the randomly chosen terminal positions. This change significantly improves connection costs and preserves the theoretical guarantee if $d_{bif} = 0$.

B. Two-level Heap Structure

As global routing graphs usually satisfy $m \in \mathcal{O}(n)$, we use binary heaps. We use one heap for each sink in S_i and a top-level heap that stores the minimum key from each sink heap. We operate with a single sink heap until the minimum label in the top-level heap is exceeded, and we extract the next minimum globally.

C. Goal-Oriented Path Searches

To speed up the path searches, we can use goal-oriented path searches, a.k.a. A^* searches [12]. This reduces the number of labeling steps. In our implementation, the connection/congestion costs between two vertices are lower bounded using landmark based future costs [11] and the delays are bounded based on L_1 -distance and the fastest layer and wire type combination for that distance.

In a path search from a sink u , we minimize the sum of these lower bounds over all targets in $S_i \cup \{r_i\} \setminus \{u\}$. When setting the connection cost of edges in a component to zero as described in III-A,

we need to make sure that the future costs stay feasible. We do this by omitting the connection future costs until every vertex of the starting component is permanently labeled and subtracting an upper bound on the connection future cost for the component containing a target $v \in S_i \cup \{r_i\} \setminus \{u\}$.

D. Better Embedding of Steiner Vertices

We can improve the position of the Steiner vertices s . After adding a path P between terminals u and v , we choose the position $\pi(s)$ on P that minimizes

$$c(E(Q)) + (w(u) + w(v))d(E(Q)) + \sum_{y \in \{u, v\}} w(y)d(E(P_{[y, s]})),$$

where Q is a cheapest s - r -path w.r.t. $c + (w(u) + w(v))d$. This corresponds to extending P to an optimum cost-distance tree for the sink set $S = \{u, v\}$. Finding the best Steiner vertex s requires an extra path search to compute Q . Instead, we use future costs to estimate it. Though, we lose the theoretical guarantees, this modification improves the quality in practice.

If the new Steiner vertex s was already permanently labeled w.r.t. l_v , we can use this path to get some further saving.

E. Encourage Root Connections

Lastly, connecting a terminal u to the root r_i early in the algorithm is very expensive because the expected node delay cost $b(u, r_i)$ reflects all delay weights. However, connecting to the root immediately reduces the cost of an optimal tree for S_{i+1} , saving a cost of at least $\eta \cdot d_{bif} \cdot w(u)$ in future iterations. We subtract these cost savings from the node delay costs for root connections to encourage merging with r_i .

IV. EXPERIMENTAL RESULTS

We integrated our cost-distance Steiner tree function as a Steiner tree oracle of a timing-constrained global routing algorithm similar to [13]. Lagrangean relaxation is used to relax global timing and routing constraints. The cost-distance Steiner tree problem (1) arises as the Lagrangean subproblem.

A. Algorithms for Comparison

We compare the new cost-distance Steiner tree algorithm with three alternatives. Each first computes a Steiner topology in the plane, considering total length instead of congestion cost. Then, this tree is embedded optimally into the global routing graph minimizing the cost-distance objective (1) using a Dijkstra-style embedding as described in [13]. The embedding greatly determines the signal speed by choosing layers and wire types.

The first routine just computes a short L_1 Steiner tree and embeds it optimally into the global routing graph.

The second is based on shallow-light Steiner trees [6], [14]. These algorithms start from an approximately minimum-length tree. During a DFS traversal, sinks are reconnected to the root whenever they violate a given delay/distance bound by more than a factor $(1 + \varepsilon)$ for an $\varepsilon > 0$. In a reverse DFS traversal, deleted edges may be re-activated to connect former predecessors if that saves cost. In [14] bifurcation penalties were also integrated for $\eta = 0.5$. We (re-)distribute them according to our more flexible model in the initial tree and during the reverse DFS traversal.

Another very popular method is the Prim-Dijkstra algorithm [2], [3]. Here, sinks are iteratively added into the root-component. A sink s and an edge e in the root component are chosen to insert a new Steiner vertex into e connecting s such that a weighted sum of total length and path length to s is minimized. Bifurcation penalties were incorporated into this method for the special case $\eta = 0.5$ [4]. We

TABLE I

AVERAGE COST INCREASE COMPARED TO MINIMUM, WITH $d_{bif} = 0$.

$ S $	#instances	L_1	SL	PD	CD
3-5	527784	0.40%	0.41%	0.44%	0.67%
6-14	156053	1.62%	1.58%	1.54%	1.57%
15-29	57594	4.70%	3.73%	3.39%	2.24%
≥ 30	84033	7.09%	5.38%	4.48%	1.73%
all	825464	1.61%	1.37%	1.27%	1.06%

TABLE II

AVERAGE COST INCREASE COMPARED TO MINIMUM, WITH $d_{bif} > 0$.

$ S $	#instances	L_1	SL	PD	CD
3-5	741967	0.96%	0.95%	1.20%	0.65%
6-14	222419	4.52%	4.32%	3.28%	1.89%
15-29	84034	11.37%	9.76%	4.82%	3.03%
≥ 30	127861	23.73%	13.39%	7.07%	3.30%
all	1176281	4.85%	3.57%	2.49%	1.34%

can distribute the delay penalty to the two branches, when selecting the edge of the root component.

Both methods use the globally optimized delay budgets that arise from the resource sharing algorithm [13].

The three methods are well established in academic and commercial design tools and form a solid foundation to compare against. They are denoted as L_1 (L_1 -shortest), SL (shallow-light), and PD (Prim-Dijkstra) in the following tables. Our new cost-distance algorithm with the practical enhancements is denoted CD.

We made tests with and without bifurcation delay penalties.

B. Results on Individual Instances

First, we present an apples-to-apples comparisons on identical cost-distance Steiner tree instances as they were generated during timing-constrained global routing on several 5nm designs, where we set $d_{bif} = 0$ in the first set of tests shown in Table I. For each instance, we measured the relative increase in the objective (1) between each algorithm and the best of the 4 algorithms. On the small instances with 3-5 sinks, the L_1 heuristic gives the best results, though results don't vary much between the 4 algorithms. Note that the embedding into the 3D global routing graph is optimum w.r.t. (1) for L_1 , SL, and PD. On larger instances with more than 15 sinks, the cost-distance algorithm dominates the other methods.

Table II shows the same results with bifurcation penalties, here the cost-distance algorithm dominates the other methods throughout.

C. Global Routing Results

Next, we compare the impact of the different algorithms on timing-constrained global routing results. Congestion is measured using the ACE [19]. $ACE(x)$ is the average congestion of the $x\%$ most critical global routing edges. We then use $ACE4 := \frac{1}{4}(ACE(.5) + ACE(1) + ACE(2) + ACE(5))$. Note that the most critical 0.5-percentile is implicitly contributing to all four numbers. An $ACE4$ of 93% is usually considered routable, but detailed routing will introduce significant detours and delays if it is above 90%.

Table III gives an overview of the instances. They are a mix of industrial microprocessor and ASIC units. Again, we start with experiments without bifurcation penalties ($d_{bif} = 0$) in Table IV. For each instance and each Steiner method, as well as for the sum/average of all instances it contains a line, and we report the worst slack (WS), total negative slack (TNS), the $ACE4$, the wire length (WL), the via count (Vias), and the wall times on an AMD EPYC 9684X processor using 16 threads. Best results are marked in green. For most

TABLE III

INSTANCE PARAMETERS (ALL ARE IN 5NM TECHNOLOGY)

Chip	# nets	# layers	Chip	# nets	# layers
c1	49 734	8	c5	420 131	9
c2	66 500	9	c6	590 060	9
c3	286 619	7	c7	650 127	15
c4	305 094	15	c8	941 271	15

TABLE IV

TIMING-CONSTRAINED GLOBAL ROUTING RESULTS WITH $d_{bif} = 0$.

Chip	Run	WS [ps]	TNS [ps]	ACE4 [%]	WL [m]	Vias [#]	Walltime [h:m:s]
c1	L_1	-49	-1 633	91.21	0.6016	561 140	00:03:24
	SL	-49	-1 544	91.32	0.6073	560 678	00:03:23
	PD	-49	-1 370	91.50	0.6104	558 133	00:05:29
	CD	-49	-1 340	89.36	0.6152	547 240	00:03:30
c2	L_1	-82	-19 774	86.36	1.0730	864 387	00:04:30
	SL	-80	-19 659	85.75	1.0838	863 571	00:04:00
	PD	-80	-19 558	85.81	1.0845	862 924	00:03:34
	CD	-80	-19 512	85.56	1.1033	854 387	00:03:52
c3	L_1	-220	-105 137	92.07	2.0070	2 336 123	00:17:33
	SL	-220	-82 142	92.28	2.0305	2 327 893	00:18:28
	PD	-220	-67 763	92.39	2.0375	2 324 940	00:20:23
	CD	-220	-67 643	91.54	2.0751	2 313 590	00:16:40
c4	L_1	-67	-31 907	90.23	6.5801	4 095 449	00:36:15
	SL	-68	-31 805	90.12	6.6114	4 083 044	00:30:07
	PD	-71	-31 769	90.21	6.6270	4 067 446	00:49:31
	CD	-68	-31 682	89.77	6.8127	4 005 655	00:43:32
c5	L_1	-276	-946 089	86.26	4.4107	4 640 578	00:21:29
	SL	-286	-869 512	86.15	4.4633	4 612 457	00:26:23
	PD	-264	-803 343	85.94	4.4687	4 580 028	00:26:58
	CD	-264	-776 373	85.27	4.6067	4 522 756	00:21:25
c6	L_1	-331	-536 208	85.96	3.6925	4 699 131	00:35:08
	SL	-314	-525 484	86.03	3.7549	4 678 851	00:43:27
	PD	-311	-509 299	86.02	3.7537	4 659 440	00:39:15
	CD	-308	-507 841	85.02	3.8492	4 613 520	00:37:18
c7	L_1	-86	-57 267	87.72	13.1894	7 817 717	01:22:23
	SL	-87	-57 903	87.69	13.2282	7 800 550	01:19:05
	PD	-90	-59 139	87.61	13.2473	7 790 466	01:18:02
	CD	-88	-54 659	87.23	13.4631	7 721 547	01:38:58
c8	L_1	-102	-174 288	91.14	13.5765	12 559 176	02:00:44
	SL	-99	-175 643	91.05	13.6386	12 519 244	02:35:41
	PD	-104	-171 323	90.99	13.6962	12 501 090	02:35:44
	CD	-105	-174 831	90.78	14.3538	12 330 992	02:33:09
all	L_1	-1 214	-1 872 304	88.87	45.1308	37 573 701	05:21:26
	SL	-1 204	-1 763 692	88.80	45.4180	37 446 288	06:20:34
	PD	-1 188	-1 663 564	88.81	45.5253	37 344 467	06:18:56
	CD	-1 181	-1 633 880	88.07	46.8791	36 909 687	06:18:24

instances our new cost-distance Steiner trees yield the best timing (WS & TNS), the best $ACE4$ and the best via count. The topology in the other routines will often force embedding path searches through congested areas, increasing their via count. However, cost-distance trees come with a higher wire length. The running time of all methods are comparable.

Table V shows results with bifurcation penalties. Note that the bifurcation penalties increase the delays and thus decrease slacks. The wire length and via count decrease compared to $d_{bif} = 0$ because the delay prices now weigh stronger compared to congestion. Again the new CD methods outperforms the others in terms of WS, TNS and via count, but has a slightly larger wire length.

V. CONCLUSION

We presented a new and faster approximation algorithm for the cost-distance Steiner tree problem, achieving the best known theoretical

TABLE V
TIMING-CONSTRAINED GLOBAL ROUTING RESULTS WITH $d_{bif} > 0$.

Chip	Run	WS [ps]	TNS [ps]	ACE4 [%]	WL [m]	Vias [#]	Walltime [h:m:s]
c1	L_1	-84	-9 837	92.15	0.5880	536 492	00:03:14
	SL	-64	-6 325	92.33	0.6039	536 398	00:03:11
	PD	-64	-5 937	92.16	0.6051	530 699	00:03:34
	CD	-61	-6 333	91.93	0.6109	526 427	00:03:09
c2	L_1	-105	-28 844	87.76	1.0728	759 747	00:03:41
	SL	-93	-22 320	85.91	1.1032	746 116	00:03:55
	PD	-88	-22 102	86.56	1.0937	733 950	00:03:31
	CD	-97	-22 022	86.07	1.1030	728 092	00:03:50
c3	L_1	-450	-2 751 437	92.33	1.9731	2 131 502	00:17:27
	SL	-315	-2 129 700	92.77	2.0701	2 107 934	00:20:58
	PD	-315	-1 831 312	92.81	2.0624	2 094 655	00:21:02
	CD	-304	-1 864 826	91.49	2.0496	2 085 304	00:17:30
c4	L_1	-84	-147 690	90.95	6.5049	3 645 350	00:31:02
	SL	-75	-135 682	90.81	6.5351	3 612 175	00:44:12
	PD	-65	-101 879	90.73	6.6586	3 533 707	00:34:26
	CD	-64	-91 618	90.62	6.7194	3 518 335	00:32:09
c5	L_1	-533	-3 654 628	87.49	4.3901	4 300 895	00:23:04
	SL	-404	-2 358 983	87.03	4.5889	4 170 442	00:32:23
	PD	-404	-2 177 493	86.75	4.5325	4 143 083	00:25:34
	CD	-415	-2 141 300	86.48	4.5952	4 093 042	00:24:21
c6	L_1	-756	-3 978 380	88.12	3.6710	4 348 194	00:35:49
	SL	-722	-3 160 751	88.94	3.8832	4 237 414	00:46:15
	PD	-698	-2 750 994	88.54	3.8960	4 185 778	00:44:29
	CD	-685	-2 703 373	87.17	3.8637	4 169 492	00:39:26
c7	L_1	-144	-339 184	88.80	13.0465	6 779 085	01:14:58
	SL	-123	-267 361	88.92	13.0961	6 740 822	01:10:15
	PD	-119	-217 997	88.72	13.2082	6 653 426	01:11:42
	CD	-120	-194 336	89.12	13.2902	6 647 930	01:34:00
c8	L_1	-197	-1 093 449	91.46	13.4100	11 450 165	02:11:43
	SL	-154	-985 353	91.59	13.5111	11 405 990	02:54:40
	PD	-114	-804 198	91.15	13.8756	11 196 187	02:44:06
	CD	-115	-804 080	91.14	14.1136	11 138 570	02:31:19
all	L_1	-2 353	-12 003 449	89.88	44.6564	33 951 430	05:20:58
	SL	-1 949	-9 066 474	89.79	45.3916	33 557 291	06:35:49
	PD	-1 867	-7 911 912	89.68	45.9321	33 071 485	06:08:24
	CD	-1 861	-7 827 888	89.25	46.3456	32 907 192	06:05:44

approximation factor of $\mathcal{O}(\log t)$. It arises as a sub-problem in timing-constrained global routing with a linear delay model (before buffering). We adopted it for better practical performance. Our experimental results show improvements over state-of-the-art algorithms on large instances and in presence of bifurcation penalties. It leads to the overall best timing and congestion results in timing-constrained global routing.

ACKNOWLEDGEMENTS

We thank Paula Heinz who implemented a preliminary version based on [17]. We reused parts her code.

REFERENCES

- [1] Albrecht, C., Kahng, A.B., Liu, B., Mandoiu, I.I. and Zelikovsky, A.Z.: *On the skew-bounded minimum-buffer routing tree problem*. IEEE TCAD 22(7), 937–945, 2003.
- [2] Alpert, C.J., Hu, T.C., Huang, J.H., Kahng, A.B. and Karger, D.: 1995. *Prim-Dijkstra tradeoffs for improved performance-driven routing tree design*. IEEE TCAD 14(7), 890–896, 1995.
- [3] Alpert, C.J., Chow, W.K., Han, K., Kahng, A.B., Li, Z., Liu, D. and Venkatesh, S.: *Prim-Dijkstra revisited: Achieving superior timing-driven routing trees*. Proc. ISPD’18, 10–17, 2018.
- [4] Bartoschek, C., Held, S., Maßberg, J., Rautenbach, D. and Vygen, J.: *Efficient generation of short and fast repeater tree topologies*. Proc. ISPD’06, 120–127, 2006.
- [5] Chekuri, C., Khanna, S., and Naor, J.: *A deterministic algorithm for the cost-distance problem*. Proc. ACM-SIAM SODA’01, 232–233, 2001.

- [6] Chen, G. and Young, E.F.: *Salt: provably good routing topology by a novel steiner shallow-light tree algorithm*. IEEE TCAD 39(6), 1217–1230, 2019.
- [7] Chuzhoy, J., Gupta, A., Naor, J., and Sinha, A.: *On the approximability of some network design problems*. ACM Transactions on Algorithms 4 (2), article 23, 2008.
- [8] Daboul, S., Held, S., Natura, B., and Rotter, D.: *Global interconnect optimization*. ACM TODAES 28(5), 1–24, 2023.
- [9] Foos, J., Held, S., and Spitzley, Y.K.D.: *Tighter Approximation for the Uniform Cost-Distance Steiner Tree Problem*. Proc. APPROX/RANDOM’23, article 19, 2023.
- [10] Fredman, M.L. and Tarjan, R.E.: *Fibonacci heaps and their uses in improved network optimization algorithms*. Journal of the ACM, 34(3), 596–615, 1987.
- [11] Goldberg, A.V., Harrelson, C.: *Computing the shortest path: A* search meets graph theory*. Proc. ACM-SIAM SODA’05, pp. 156–165, 2005.
- [12] Hart, P.E., Nilsson, N.J. and Raphael, B.: *A formal basis for the heuristic determination of minimum cost paths*. IEEE TSSC 4(2), 100–107, 1968.
- [13] Held, S., Müller, D., Rotter, D., Scheifele, R., Traub, V. and Vygen, J.: *Global Routing With Timing Constraints*. IEEE TCAD 37(2), 406–419, 2018.
- [14] Held, S. and Rotter, D.: *Shallow-Light Steiner Arborescences with Vertex Delays*. Proc. IPCO 2013, 229–241.
- [15] Hu, J., Sapatnekar, S.S.: *A timing-constrained simultaneous global routing algorithm*. IEEE TCAD 21, 1025–1036, 2002.
- [16] Kruskal, J.B.: *On the shortest spanning subtree of a graph and the traveling salesman problem*. Proc. AMS 7(1), 48–50, 1956.
- [17] Meyerson, A., Munagala, K., and Plotkin, S.: *Cost-distance: Two metric network design*. SIAM Journal on Computing 38(4), 1648–1659, 2008.
- [18] Wei, Y., Li, Z., Sze, C., Hu, S., Alpert, C.J. and Sapatnekar, S.S.: *CATALYST: Planning layer directives for effective design closure*. Proc. DATE’13, 1873–1878, 2013.
- [19] Wei, Y., Sze, C., Viswanathan, N., Li, Z., Alpert, C.J., Reddy, L., Huber, A.D., Téllez, G.E., Keller, D. and Sapatnekar, S.S.: *Techniques for scalable and effective routability evaluation*. ACM TODAES 19(2), 17:1–17:37, 2014.
- [20] Yan, J.-T., Lee, C.-F., Chen, Y.-H.: *Multilevel timing- constrained full-chip routing in hierarchical quad-grid model*. Proc. ISCAS, 5439–5442, 2006.
- [21] Yang, J. and Kim, T.: *Improving Timing Quality Through Net Topology Optimization in Global Routing*. Proc. 37th SOCC, 1–6, 2024.
- [22] Yang, L., Sun, G. and Ding, H.: *Towards Timing-Driven Routing: An Efficient Learning Based Geometric Approach*. Proc. ICCAD’23, 1–9, 2023.